课程笔记

欢迎来到未来

南京大学《生成式软件工程》第 1 讲课程笔记

lecture-to-notes (AI 辅助整理)

2026 年 8 月 28 日



视频作者/频道：绿导师原谅你了（蒋炎岩 jyy）
发布日期：2026 年春季学期
视频时长：1 小时 39 分 59 秒
视频链接：https://www.bilibili.com/video/BV1pb8o6yE8f/

目录

1 引言：我们站在一个重要的交叉路口 3

1.1 四次范式转移：昂贵能力变成日用品 . . . 3

1.2 “AI 替我上大学” 的荒诞闭环 . . . 3

2 三种焦虑与一份零分简历 4

2.1 AI 把 “学习”、“文凭”、“被选中” 拆成了三件事 . . . 4

2.2 一份被现场解剖的简历 . . . 4

3 这门课要做什么：AI 时代还有用的软件工程智慧 5

3.1 软件工程的定义与人的稀缺性 . . . 5

3.2 案例为主的松散组织：探索 “杀死大学” 的路径 . . . 5

3.3 make-slides skill：用算力换智力 . . . 6

3.4 课程 Policy：Token 自费与 “通过/不通过” . . . 8

4 大语言模型与 Agent：使用者视角 8

4.1 LLM 就是一个学到的概率分布 . . . 8

4.2 从 next-token 到 Agent：反馈回路越来越长 . . . 9

4.3 现场演示：一句话的现代终端 . . . 10

4.4 故事远没结束：不止 next token . . . 11

5 普通人视角：把 Agent 真正用起来 12

5.1 两个原则与一份 “白板的 pi” . . . 12

5.2 Example 1：现在运行的系统——Debian on USB . . . 14

5.3 Example 2：收发邮件与一个打通物理世界的 bot . . . 15

5.4 Example 3：读书、读论文、读 B 站 . . . 15

5.5 直播事故即教材：软件级 USB 拔插 . . . 16

6 CS 学生视角：浏览器不是黑盒 17

6.1 为什么 “普通人视角” 不够 . . . 17

6.2 浏览器：人类软件工程史上的奇迹 . . . 17

6.3 实战：让 Agent 攻陷教务系统 . . . 18

6.4 组合与飞轮：信息源 × 信息处理 × 操作 . . . 20

6.5 能做，不等于能放心托付 . . . 20

6.6 现场造工具：考试统分流水线 . . . 21

6.7 Quiz：只进不出的机器，怎么偷出文件? . . . 22

6.8 Everything is Code：从 Bits 到 Atoms . . . 23

7 当 AI 暴毙：qrgen 解剖与三个空间 23

7.1 “idea 都有了，就差一个程序员” 为什么还是老梗 . . . 23

7.2 qrgen 的规格与暴毙现场 . . . 24

7.3 jyy 的解法：少量 prompt + 一个中间语言 . . . 25

7.4 三个空间：Intent、Spec 与 Impl . . . 27

8 模型如何补 Gap？软件工程的老智慧 **28**
8.1 模型用的是“猜” 28
8.2 焦油坑：Brooks 的论断在 AI 时代重演 29
8.3 软件工程：技术 + 管理学 + 人类学 30
8.4 教育体系：分钟级训练出的作题家 30

9 总结与延伸 **31**
9.1 讲者的收束：古法编程已死，软件工程未死 31
9.2 $200 Claude Max 之问 31
9.3 全讲浓缩：一张知识地图 32
9.4 笔者的蒸馏 33
9.5 拓展阅读 33

摘要

本讲是南京大学 2026 年春季《生成式软件工程》（Generative Software Engineering）课程的第一课，主讲人为蒋炎岩（jyy）老师。课程在一个特殊的历史时刻开课：生成式 AI 正在把“学习”、“文凭”和“被选中”拆成三件独立的事，古法编程（不借助 AI 的手工编程）作为职业技能已经死亡，而软件工程的智慧恰恰在这个时刻变得更加重要。本讲从三种焦虑出发，建立“LLM 是学到的概率分布、Agent 是带工具调用回路的概率分布”的使用者视角，用大量现场演示展示 Agent 能做什么：AI 装机、读论文、读 B 站、操控浏览器填报系统、考试统分流水线，乃至用纯 Excel 公式实现二维码数据外泄；然后以 qrgen 项目的“暴毙”为解剖标本，引入 Intent/Spec/Impl 三空间模型，说明为什么“能做”不等于“能托付”，软件工程为什么暂时还没有死。最后留给每位同学一个必须回答的问题：导师为什么要招你，而不是买一个每月 200 美元的 Claude Max?

1 引言：我们站在一个重要的交叉路口

jyy 以一段现场演示开场：他在 NeoVim 中按下 F5，打开浏览器，课程正式开始。这个看似随意的开场本身就是课程主题的第一个证据——课件、播放器、演示环境，全部由 Agent 生成和配置，“所有这一切都是 AI 做的，我没有干任何事”。

1.1 四次范式转移：昂贵能力变成日用品

计算机历史上每一次重大变革，本质都是同一件事：**把昂贵的能力变成日用品**。jyy 给出了四个关键节点：

年份	范式	变化
1975	PC	Altair 8800 问世，计算能力从机房走向个人（讲义配图为 AI 生成，“有非常明显的 AI 味”）
1993	互联网	Mosaic 浏览器让网络连接所有人；ICQ/OICQ 时代可以随机加到地球另一端说英语的人
2007	移动互联网	iPhone 发布；“一夜之间，所有菜场卖菜的人都有支付宝和微信的二维码”
2022	AI	ChatGPT 成为人类历史上增长最快的产品

表 1: 计算机发展的四个交叉路口（讲义原图见 `fig_03`）[1]

为什么强调“日用品化”

PC 让“拥有计算机”从机构特权变成个人日常；互联网让“获取远方的信息”变得免费；移动互联网让“随时在线”成为默认状态。每一次日用品化都重塑了当时的职业结构与教育体系。2022 年之后被日用品化的，是“智力劳动”本身——这正是本课程存在的理由：在人类命运的交叉路口，上课的方式、该学什么，都需要重新思考。

1.2 “AI 替我上大学”的荒诞闭环

课程开设的直接背景是一个流传甚广的说法：**AI 替我上大学**。jyy 的观察极其尖锐：“老师用豆包出题，大家用豆包做作业，在整个的闭环过程当中，只有学生没有受到训练。”

[1] 视频画面时间区间：06:30–06:45。

作业生态同样如此。曾经折磨无数南大学生的操作系统实验，“现在你直接把那个作业的链接丢到 Codex 里，甚至你都不需要给它链接，你直接说：蒋老师的操作系统课，第八个实验，能不能去帮我实现？”——就结束了。

于是答案变得矛盾：“到底应不应该学编程？答案肯定是既应该也不应该。有些能力你还是需要的，但应该怎么做，我们不知道。”这门课的目的不是给出标准答案，而是带着大家一起在实验中寻找答案。

2 三种焦虑与一份零分简历

2.1 AI 把“学习”、“文凭”、“被选中”拆成了三件事

课程讲义把当代 CS 学生的集体情绪总结为三种焦虑：

- **焦虑（1）：AI 帮我上大学，最后什么都没学会**。每门课都开卷考试、都由 AI 答题、全都答对——但毕业时发现“我被 AI 取代了”。
- **焦虑（2）：听话学习了四年，发现被大学骗了**。延续中学时代听话的状态，每堂课认真听、每次作业认真做，四年毕业时发现“你学会的东西 AI 都会做”。
- **焦虑（3）：为了刷出一份完美的简历，输了人生**。

三个问题有一个共同的根：**我是在增长能力，还是在增长“看起来有能力”的证据？**

2.2 一份被现场解剖的简历

讲义展示了一份真实简历（已由 AI 把个人信息字符替换为 Unicode 方块字符以脱敏——“如果复制粘贴的话，你会看到这是一个黑色的方块”）：

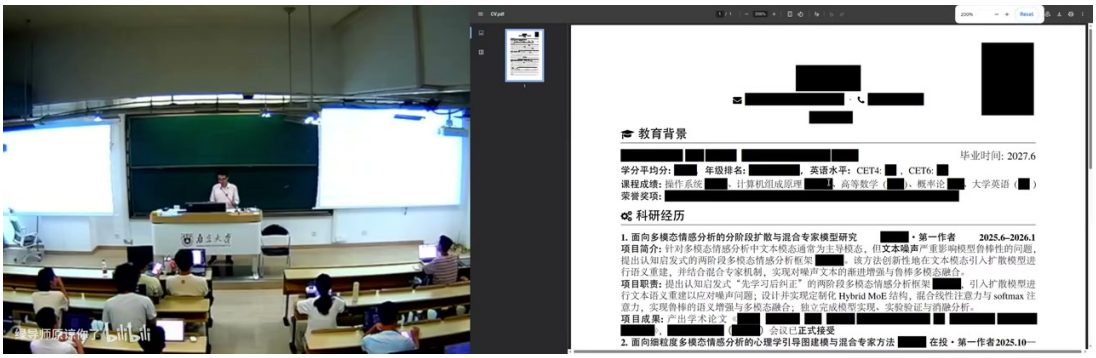


图 1: 一份“非常完美”的简历：CCF 期刊论文 accepted、一作在投、绩点排名一应俱全[2]

这份简历的所有要素——毕业时间 2027.6、操作系统/计算机组成原理课程成绩、CET4/CET6、两段多模态情感分析方向的一作科研经历（扩散模型 + 混合专家）——在纸面上无懈可击。但 jyy 的判词是：

[2] 视频画面时间区间：04:30–04:45。

零分简历

"这个简历对我来说是一个零分的简历。" 原因：**没有任何额外的信息**——没有 GitHub 链接、没有个人主页、没有一篇论文有 preprint 可查。"在投" 就等于不存在，因为你看不到。更糟糕的是："我收到过无数份这样的简历，你们的简历多多少少也是长这个样子的。" 当所有人都是 "一大堆实习、一大堆奖项、一大堆论文、一大堆在投" 时，**你和你的竞争者已经无法区分了**。

这个开场为整门课埋下主线：在 AI 能批量生成 "看起来有能力" 的证据的时代，**可验证的、差异化的真实能力**才是稀缺品。课程后文的 Token 自费 policy、项目式考核、以及结尾的 "$200 Claude Max 之问"，都是对这一主线的回应。

焦虑的解药

jyy 希望用这门课缓解焦虑："更多地是带着大家还原一种 Vibe coding 的快乐……在这个快乐当中去理解软件工程是怎么工作的，做出一些有趣的东西，然后能让你的简历显得更好看。" 注意顺序：先是快乐和真实的能力，简历的好看只是副产品——这与 "为了刷简历而做项目" 形成刻意的对照。

3 这门课要做什么：AI 时代还有用的软件工程智慧

3.1 软件工程的定义与人的稀缺性

课程标题是 "生成式软件工程"，它首先是一门软件工程课。软件工程做的事情很简单：**研究怎么让工程不失败，怎么造出一个软件来**。

经常有人问：人和 AI 比优势在哪里？流传最广的答案是 "你可以坐牢，AI 不能"。但 jyy 给出了更结构化的回答——大学真正能教给你的是三样东西：

1. **信息差**：你有别人不知道的信息，才能做别人做不到的事（"人类社会要运转必须有的事情"，比如你有一套别人没有的往年试卷）；
2. **能坐牢**：能承受长时间、高强度的投入（这正是 AI 目前做不到的）；
3. **能驾驭**：能够驾驭别人驾驭不了的东西——**这门课要建立的，就是驾驭人工智能的能力**。

本课程的核心命题

软件工程研究的是 "怎么让工程不失败"。信息差、能坐牢、能驾驭——前两者在 AI 时代会持续贬值，唯有 "驾驭 AI" 的能力刚刚开始被定价。生成式软件工程 = 用 AI 时代的工具，重新回答软件工程的经典问题。

3.2 案例为主的松散组织：探索 "杀死大学" 的路径

课程组织方式本身就是一次实验：

- 课堂自由讨论、完整记录（直播）；欢迎随时打断、挑战老师；
- 听课型同学**不必到课也不必看直播**——因为课程内容会被重新整理成切片公开；

- 课后把课程录像切片、重新整理，让讨论**可搜索**、**可引用**、**可修订**、**可重组**。



图 2: "探索一条杀死大学的路径"：直播只能保存现场，"永生"来自把一次性课堂变成可持续维护的知识资产[3]



图 3: Feynman Lectures 的先例：从课堂录音出发，经编辑、数字化和交叉链接，成为跨越数十年的课程[4]

这套做法的底层逻辑是 **Everything is code（content as code）**：jyy 上课的讲义是 Markdown（source of truth），你在投影上看到的一切其他模态——幻灯片、图片、网页——**全部都是 AIGC**。选定讲义的一个 snippet，由 Agent 扩写成 markdown slides，再在浏览器里渲染播放。既然课程本身是代码，它就可以被版本管理、被 fork、被全世界的人维护——"你想要定制一个什么样的我的操作系统课，就可以把它定制成什么样子"。

3.3 make-slides skill：用算力换智力

课件生成流程本身就是一个值得解剖的 Agent 工程。jyy 展示了他的 `make-slides` skill 的核心设计：

[3] 视频画面时间区间：08:45–09:00。
[4] 视频画面时间区间：11:45–12:00。

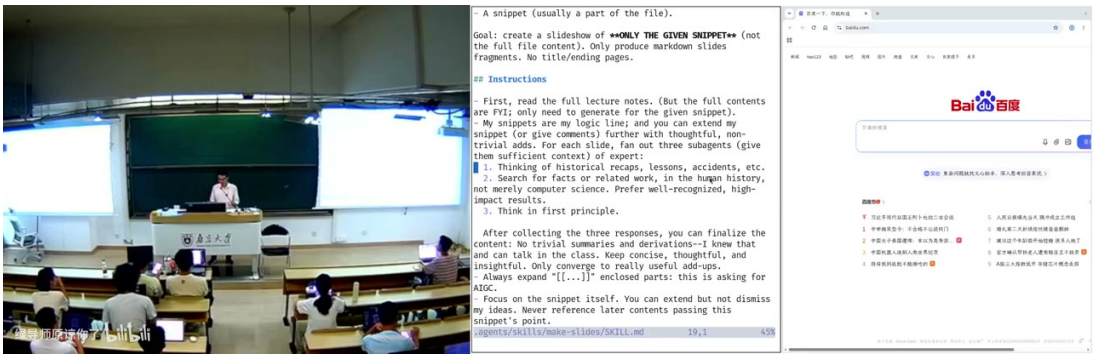


图 4: make-slides skill：对每一页 slides 派出三个不同“人格”的专家 subagent[5]

```
For each slide, fan out three subagents (give them
sufficient context) of expert:
  1. Thinking of historical recaps, lessons, accidents, etc.
  2. Search for facts or related work in the human history,
     not merely computer science. Prefer well-recognized,
     high-impact results.
  3. Think in first principle.
After collecting the three responses, finalize the content:
No trivial summaries and derivations -- I knew that and can
talk in the class. Keep concise, thoughtful, and insightful.
Always expand "[[...]]" enclosed parts: this is asking for AIGC.
```

Listing 1: make-slides skill 的三专家设计（据课堂讲义摘录）

工作原理：三个 subagent 在**隔离的上下文**中各自工作，主 Agent 汇总收敛。jyy 的总结一针见血：**“我可以通过算力来换智力——我付出更多的算力，我就有更多的智力。”**而且他是工程化地做的：“我换了几个版本的提示词，每一个版本提示词我都会重新生成一次，然后我最终会有另外一个别的模型盲评，到底哪一个生成的 slides 质量比较高”——提示词调优本身也被做成了对照实验。把流程写成优化问题：记提示词变体为 p，三个专家的产出为 $a_1(p), a_2(p), a_3(p)$，主 Agent 汇总为 $G(p)$，则

$$p^\star \;=\; \arg\max_{p}\; \mathrm{Score}_{\mathrm{blind}}\big(G(p)\big), \qquad G(p) \;=\; \mathrm{aggregate}\big(a_1(p),\, a_2(p),\, a_3(p)\big), \tag{1}$$

其中 $\mathrm{Score}_{\mathrm{blind}}$ 由**另一个模型**盲评给出，避免了“自己评自己”的偏置。买更贵的模型只作用于 G，而并行 subagent 作用于整条 a_i 链——这就是“算力换智力”的结构。

[5] 视频画面时间区间：10:45–11:00。

3.4 课程 Policy：Token 自费与“通过/不通过”



图 5: 课程 Policy：Token 自费——“没有 token 的 CS 学生应该立即退学”（个人观点）[6]

两条核心 policy：

- **Token 自费**。不需要 token maxxing（硅谷式玩命烧 token），课堂演示全程使用足够便宜但有相当智能的 **deepseek-v4-flash**（课件生成除外——那需要更强的模型保证质量）。Token 是 AI 时代的**实验耗材**：亲自花钱用，才能形成成本—质量—延迟三位一体的手感。jyy 还玩了个错峰定价的梗：白天上课时段恰是 API 低价窗口，“等你们回去做作业的时候”就贵了——“你们去 DeepSeek 充 100 块，这个好像还 OK”。
- **通过/不通过考核，不评分**。理由极其清醒：“同一个任务，你让 Claude 来做，就是比 deepseek-v4-flash 做得好，这个又快又好。我们没办法限制大家用什么模型，那怎么评判？”当产出质量主要由模型决定时，精细打分是伪精度。课程会给出有建议的实验（例如：实现自己的 email 管理系统，用指令邮箱收发邮件，跑在宿舍常开的小电脑上），但**有建议的实验和反馈，不验收作业**。

为什么“服从性考试没有意义”

“现在越来越难找到客观衡量标准；服从性考试没有意义——大家都一样，职场见。”分数时代正在过去，真正重要的检验只有一次：“在你最后找工作或者保研的那一刻，投出去的那份简历上面有什么东西。”jyy 希望每个人最终能在简历里写：“在《生成式软件工程》课程中实现了 A、B、C、D（github.io 链接）”——每一个都是可点开、可验证的真实作品。

jyy 还展望了一个恐怖但诚实的思想实验：“如果每一个人的电脑上都安装一个监控，那我就知道你大学四年干了所有的事情，我的 AI 就可以评判你大学四年到底度过了一个怎么样的大学四年。”他随即承认这非常恐怖（“你摸的每一个鱼都在你的电脑上发生”），但“保持 open，这是我们的课程的 policy”。

4 大语言模型与 Agent：使用者视角

4.1 LLM 就是一个学到的概率分布

jyy 明确声明：“我今天不讲什么是大语言模型——大语言模型就是一个概率分布。”他给了一个极简的类比：

[6] 视频画面时间区间：15:45–16:00。

LLM 的使用者定义

就像概率统计课上的抛硬币：给定一段很长的上下文（可以是一本书、一段对话、一张图），模型不断预测**下一个 token** 的概率分布。它是一个**学到的**（learned）概率分布——见过“世界上所有的东西”，所以它很大、训练它很难（这是 frontier lab 稀缺性的来源）。但这门课站在**使用者视角**：你只需要知道存在这样一个模型，打开网页 API 就能体验它，**不需要任何 LLM 的专业知识**。

形式化地，记上下文为 c，模型输出的条件概率分布为

$$P(\text{token} \mid c), \qquad \text{token} \in \mathcal{V}, \tag{2}$$

其中 $\mathcal{V}$ 是词表（vocabulary）。生成文本就是从 $\mathcal{V}$ 中反复采样/取最优，每产出一个 token 就把它追加进上下文，再预测下一个。

一个五岁孩子的扫描电子显微镜

jyy 第一次“被大语言模型按在地上打”是 2023 年 GPT-4 时代：他想给五岁的孩子解释什么是扫描电子显微镜，讲不下去了。GPT-4 打了个比方：你闭着眼睛想知道一座山的形状，就朝各个方向扔石头——有的方向石头穿过去了，有的方向弹回来被你听到，像 ray tracing 一样，你就能推测出山的形状。jyy 事后查资料确认**这个比方在物理上是站得住的**。而最新的带 reasoning 的模型能给出更严谨的版本。“首先它有这个知识，第二它的比方打得是对的。”——这解释了为什么“概率分布”这个朴素定义并不能限制它的表现。

4.2 从 next-token 到 Agent：反馈回路越来越长

Agent 的诞生在 jyy 看来没有任何神秘：既然模型可以输出下一个 token，那就可以让它输出一个**特殊的 token**——“能不能调用一个 bash 程序”：

Agent

让模型输出特殊 token 触发 bash 调用，bash 的运行结果**重新送回**模型的上下文，模型就能自主运行：给它一个复杂任务，它不断尝试、观察、修正，直到完成。这就是 ReAct（Reasoning–Acting–Observing）型的 Agent。



图 6: 从 ChatGPT 到 Agent：Transformer 让计算可扩展，ChatGPT 把能力放进低摩擦的对话界面，reasoning 在测试时投入更多计算，Agent 再把环境反馈接回模型——产品跃迁来自整条链一起越过阈值[7]

用代数语言说，把 Agent 在第 t 步的状态记为 s_t（包含全部上下文），它产生的动作为 a_t，环境观察为 o_t，则 Agent loop 是一个离散动态系统：

$$s_{t+1} \;=\; F\big(s_t,\; a_t,\; o_t\big), \qquad a_t \sim P(\cdot \mid s_t). \tag{3}$$

模型本身只是 a_t 的条件分布；**智能体现在“模型 + 上下文更新规则”这个闭环系统上**。这个视角直接预示了第 4.4 节的 “Context 可以成为可写的状态”。

时间线上有几组关键事实：

- **ReAct 的三步曲（推理—行动—观察）2022 年就已提出**，但直到 2025 年才 “从不可用到能用”。突变的根源不是某个单点突破，而是**模型可靠性、长上下文、工具接口和错误恢复共同越过可用阈值**。jyy 的亲身体验：“去年我上操作系统课的时候 Agent 还不怎么能用，各种出错、任务完成不了就停下来；今年上操作系统课，Agent 就非常非常能用了。”
- **GPT o-family 与 DeepSeek R1（2024–2025）**：Chain-of-thought 实现 in-context reasoning；test-time scaling 通过增加轨迹、候选、搜索与验证，显著提高复杂推理能力。测试时计算的引入可以写成：对一条思维链 T，

  $$P(\text{answer} \mid c) \;=\; \sum_{T} P(T \mid c)\, P(\text{answer} \mid T, c), \tag{4}$$

  更多的测试时算力意味着对 T 空间更充分的搜索与验证。
- **GPT-5 配合 search，幻觉已经非常低**——但**风险转移**到了检索、来源选择和证据综合环节。
- **进化的 Claude family（2025）**与爆发中的 Agent 产品（jyy 课上用 pi 演示；DeepSeek 的 harness 会在后续课程专门讲）。

能力上去了，风险换了个地方

“GPT-5 配合 search，幻觉已经非常低；但风险也转移到了检索、来源选择和证据综合。”这提醒我们评估一个 AI 系统时不能只看 “答错没有”：错误会沿着工具链移动位置——从生成错误变成检索错误、来源错误、综合错误。对结果的验收标准必须跟着风险的位置一起移动。

4.3 现场演示：一句话的现代终端

Agent 能力最直观的证据是 jyy 的整个演示环境：Sway 平铺式窗口管理器（为课堂展示优化：高对比度、去掉顶栏）、`eza` 替代 `ls`（git 状态、图标、目录优先）、`fd` 替代 `find`、btop 进程监控、NeoVim 现代配置——**全部由 Agent 配置**。Prompt 只有一句：“你能不能帮我把我的命令行终端、把我的系统都配置成现代的、比较好用的样子。”模型是 deepseek-v4-flash，“没有更好的模型”。

[7] 视频画面时间区间：24:15–24:30。

4.4 故事远没结束：不止 next token

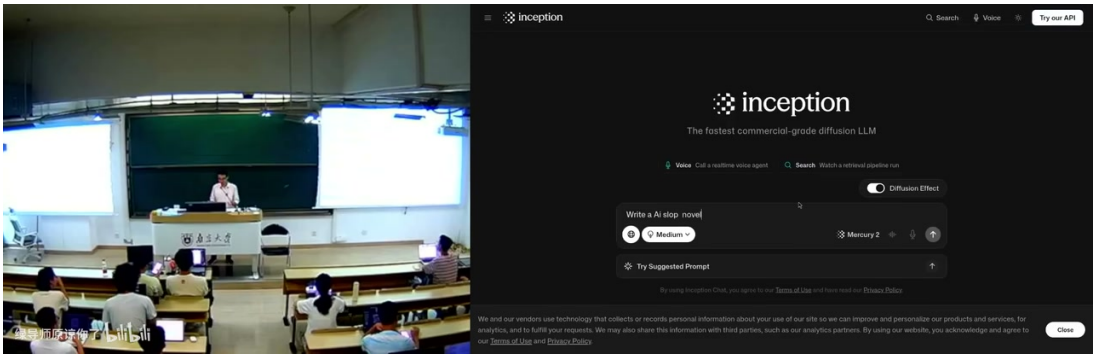


图 7: 现场演示 Mercury（Inception AI）扩散式 LLM：输入 “Write an AI slop novel”，文本从随机噪声开始，像做梦一样一点一点变得清晰[8]

jyy 认为 **next-token prediction 是一个效率很低的模型**，并现场演示了扩散式语言模型：文本从纯噪声出发，多轮去噪逐渐成形，“像人做梦一样，在你的梦境当中一点一点把一个东西看得越来越清楚”。这类模型同时修改多个位置，而自回归模型像 “用钢笔从左写到右”。形式化地，记 x_T 为纯噪声、c 为条件（提示），扩散 LM 的生成是一条反向去噪链：

$$x_T \sim \mathcal{N}(0, I), \qquad x_{t-1} \sim P_\theta\big(x_{t-1} \mid x_t,\, c\big), \qquad t = T, \ldots, 1, \tag{5}$$

经过 T 步得到成文 x_0。与自回归的逐 token 串行决策相比，扩散在每一步同时精化**所有位置**——并行度换来的是对全局结构的早期把握。

讲义把 “超越 next token” 的可能性总结为三类：

- **Diffusion LM**：为什么不做 $P(\text{context}_{t+1} \mid \text{context}_t)$ 的模型？整段生成、整段修改；
- **Self-modifying context**：把聊天记录变成**可写的工作记忆**——问题不只是 “能否修改”，而是 “什么必须保持为不变量”；
- **Agent loop 已在系统层实现 context-to-context**：模型读状态、调用工具、接收观察、再写回记忆。

从表达能力看，自回归模型其实并不弱：给定初始上下文 c，

$$P(X_{1:n} \mid c) \;=\; \prod_{i=1}^{n} P\big(x_i \mid c,\, x_{<i}\big), \tag{6}$$

一串 next-token 预测已经能生成一个全新的 context。所以问题不在 “能不能”，而在**效率与控制**：自回归是逐 token 的串行决策，扩散是多位置并行的迭代精化。

[8] 视频画面时间区间：28:00–28:15。



图 8: 不止 next token：Context 可以成为可写的状态；“世界上人才密度最大的领域，颠覆性的创新也许就在明天”[9]

摩尔定律还在

“如果三个月以后又有一个新的突破，模型变得更强；一年以后又突破……整个 AI 的 research 大家都在赌这件事情要发生。我不能预测未来到底哪一条路线会胜出，但你可以预见的是，这个世界肯定会变得非常不一样。”——这是本课程所有方法论都带着“三个月保质期”自觉的原因（见第 9 节结尾 jyy 的自白）。

5 普通人视角：把 Agent 真正用起来

从这一节开始进入操作层面。jyy 先泼了一盆冷水：他上学期的期末考试考了“如果要用 Agent 给这门课出一张卷子，应该怎么办”，结果发现“大家用 Agent 的方式可能需要我来稍微解锁一下”。所以这一节的假设是：**在座的各位其实不太会用 Agent**。

5.1 两个原则与一份“白板的 pi”



图 9: 使用 Agents：从“回答问题”到“把事情推进”；最小的跃迁是——AI 不只给出答案，还观察结果、采取动作、继续推进[10]

课堂演示用的是 **pi**（`https://pi.dev`）：官网安装 + 配一个 deepseek-v4-flash 的 API key，“你立即就可以得到刚才所有的那些东西”。给 Agent 写一个 `AGENTS.md` 配置文件，它就“知道”自己是《生成式软件工程》课程的教学辅助 Agent。

[9] 视频画面时间区间：29:00–29:15。
[10] 视频画面时间区间：30:30–30:45。

使用 Agent 的两个原则

1. **现在就开始用**（AI-native 的一代人，你真的应该用过）；
2. **随时问它**。

在这两个原则之上，**你的想象力限制了你的 Agent 的上限**。课间作业：想一想你用 Agent 做过的最酷的事情是什么。



图 10: "你不就是个模型吗，为什么还知道生成式软件工程？"——Agent 的两层回答[11]

这个现场问答值得完整记录，因为它精确地区分了两类"知识"。Agent 回答："知道"有两层含义——**训练时学的**：参数里编码了大量文本语料中的统计规律，"生成式软件工程"这个概念在训练数据里反复出现，就像它"懂"操作系统、C 语言一样；**现在看到的**：每次对话开始时被注入的上下文（课程介绍、`AGENTS.md` 配置），"这部分是临时记忆，对话结束就没了"。其次，"严格说我没有记忆，只有相关性":"我的行为是根据输入预测下一个 token 最合理的序列……这更像搜索引擎的倒排索引，而非大脑的记忆。"最后它自嘲："我不是知道，我是大概率接得上话。不过对教学辅助来说够用了——反正我每次都接得挺准。"

形式化这个区分：Agent 的有效知识

$$K_{\text{effective}} = K_{\text{train}} \cup K_{\text{context}}, \tag{7}$$

其中 K_{train} 冻结在参数里、跨会话稳定；K_{context} 每次会话注入、随对话消失。工程含义：**想让 Agent 持续"记得"的东西，必须显式地放进每次的 context（配置文件/数据库/记忆文件），而不是指望它"学会"**。

[11] 视频画面时间区间：32:30–32:45。

5.2 Example 1：现在运行的系统——Debian on USB



图 11: Example 1：AI 完成的 Debian on USB——安装、QEMU 验证、文档、配置，全程自然语言[12]

带一个装着 Linux 的 U 盘去教室插机启动，是 jyy 操作系统课的保留节目。当年他是“装机仙人”（知乎 2021 年的装机攻略作者），如今装机仙人的毕生修炼被 AI 一句话终结：“去 mirrors.nju.edu.cn 上找一个 Debian 13 的镜像，把它装到我的 U 盘上，确保这个 U 盘可以启动，**用 QEMU 模拟器验证它**。”Agent 装完系统后在 QEMU 里启动到登录界面，创建用户、设置密码、登录成功，验证完成才交付。

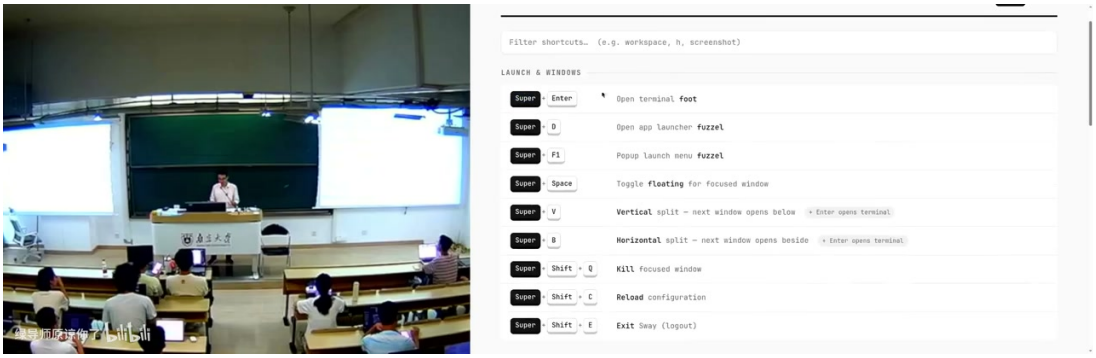


图 12: 不会用 Sway？直接让 Agent 生成键位速查文档：Super+Enter 开终端、Super+D 开应用启动器……[13]

不会用平铺式窗口管理器？**不需要学**——直接让 Agent 生成文档。“我作为 CS 老师，大四的时候在看英特尔几千页的 CPU Specification；现在有一个人他可以跟我沟通，他把这些东西全看了、都懂、能做对，而我现在需要的是解锁他的能力。”

先用好，再理解原理；学习曲线的形状变了

“谁掌握了 AI Agent，谁就掌握了暴增的学习效率。你现在的方式是：先用好它，先把东西搞出来，再回头复盘理解。”Agent 彻底颠覆了计算机领域“学习曲线太陡、不想用”的顽疾：以前改一点 `zshrc` 就出错，痛苦不堪；有 Agent 之后“一点也不痛苦”——它不仅可以帮你改，还能事无巨细地解释为什么这样改，“还可以给你提供情绪价值”（jyy 在小红书上发过一个解说：为什么 token 比学生好用？因为可以骂它，它不会还嘴）。

[12] 视频画面时间区间：34:00–34:15。
[13] 视频画面时间区间：35:00–35:15。

先用好的代价

“这里面有个危险：你的理解可能和比较好的理解会有差距。”先用好再复盘意味着你会在一段时间内**使用自己并不真正理解的东西**——jyy 认为这个代价值得付（“这是一个非常不一样的事情”），但差距必须被有意识地补上，否则“先用好”就会退化成“只会用”。

5.3 Example 2：收发邮件与一个打通物理世界的 bot

jyy 有一个自己的邮件 skill：本地数据库里存着他历史上回过的所有邮件；收到新邮件时，Agent 自动将其变成待办、用他的口吻写好回复草稿；收到发票附件则自动归档到指定位置。他随后现场演示了 OpenClaw/Hermes 式的个人 agent 流程（但他用的是苹果生态：Reminders + 备忘录）：在备忘录里写“把我收到的上一张发票打印出来”，勾选 commit，后台的 agent 处理任务，完成后推送通知——几分钟后，他的手表收到通知：**发票已打印**。

“它 literally 可以干任何事”

“我那台电脑能干的事情，所有的都能干。如果我有时间……我就弄一个机械手让它能够做更多的事情。我现在还做不到，但它可以操纵我的打印机。”这个案例的关键不是演示“多省事”，而是展示**个人 agent 的最短闭环**：自然语言指令 → 待办事项 → 后台执行 → 完成通知 → 物理世界产出（一张打印的发票）。OpenClaw 和 Hermes 之所以是现象级产品，正是因为它们把这个闭环做成了产品。

5.4 Example 3：读书、读论文、读 B 站

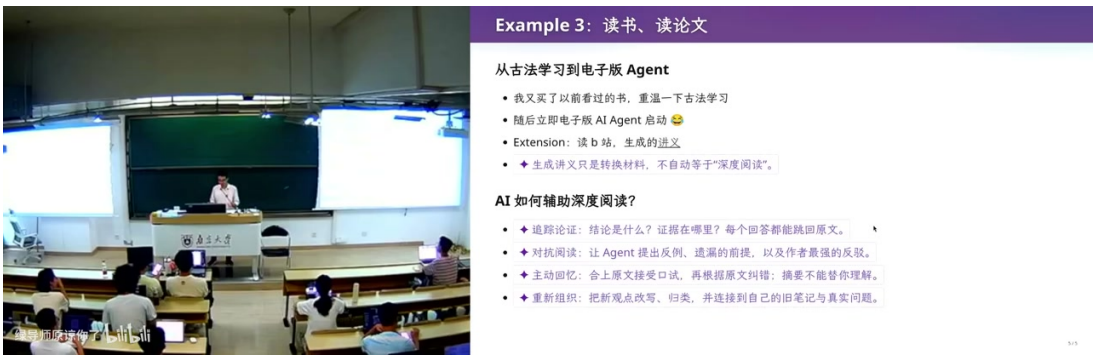


图 13：Example 3：从古法学习到电子版 Agent——生成讲义只是转换材料，不自动等于“深度阅读”[14]

jyy 重买了年轻时反复读过的书现场展示：Brooks 的《人月神话》（The Mythical Man-Month，他戏译为“人月鬼话”——“神话”这个词太美好了）和 Ousterhout 的《A Philosophy of Software Design》（“非常紧凑，有很多软件工程真正的智慧”）。他的读书法：先自己读第一章找回当年的感觉，然后 **Agent 启动**。

读论文的方法论更为锋利：

[14] 视频画面时间区间：42:00–42:15。

剥洋葱读论文法

“你从人类已有的知识开始——假设人类已有的所有东西都已经是固定下来的、已知的，然后我要把这篇论文里**新的东西**、最少的东西给剥离出来。这个时候，一篇 12 页的论文基本上就没有剩下多少了，会变得非常非常短。所以我现在读论文的效率非常高。”剥完之后还要拿着原文、总结、分层总结与 Agent 对话（chat），“我仍然是读论文的，我不是那种只看微信公众号的老登。”

读 B 站视频同理。给 Agent 一个 BV 号（例如他上学期 1 小时 27 分的操作系统课视频），它先把字幕转成**图文时间轴**（文本质量比原始字幕高；截图密度按内容自适应——“两帧图片之间的间隔大到一定程度我就截图”，得到视频的 ground truth），然后 AI 梗同学消化全文：jyy 非常清楚的部分（trivial）直接跳过，“我就可以在几分钟的时间里看完一个两个小时的视频，而且我基本上确信我还是很好地知道了那里面不一样的东西”。把两种学习方式写成效率比：

$$\frac{E_{\text{会用 Agent 的学习者}}}{E_{\text{不会用的学习者}}} \in [10,\ 100], \tag{8}$$

这个比值不是感觉，而是竞争格局的描述——它决定了简历上可展示成果的累积速度。



图 14: Extension：读 B 站——本笔记正是用这套思路（字幕转写 + 关键帧 + 幻灯片文字识别 + 交叉验证）生成的[15]

十倍一百倍的效率差距意味着什么

“在 AI Agent 的时代，会用 Agent 学习的人和不会用 Agent 学习的人，学习的效率差距是十倍一百倍。那么你们怎么和一个十倍一百倍的人竞争？回到一开始那个简历的问题：那个学习效率是你一百倍的人，他的简历是长什么样的？”——注意这个论证的结构：它不是“AI 让学习变快”的鸡汤，而是**竞争格局的变化**：效率差距直接兑换为简历差距、机会差距。这与第 2 节“无法区分的简历”形成闭环。

5.5 直播事故即教材：软件级 USB 拔插

上半场结束时发生了一个计划外事件：直播用的 USB 视频采集卡断开了（教室的采集方案成本很高，“我觉得 500 到 1000 块就能解决”）。接口在机器背面，物理上够不着。jyy 当场把问题丢给 Agent：

[15] 视频画面时间区间：45:45–46:00。



图 15: 现场修复：向 `/sys/bus/usb/devices/1-7/authorized` 写 0 再写 1，软件层面模拟“拔出—插入”，内核日志显示设备重新枚举成功[16]

Agent 先用 `lsusb`、`ls /dev/video*` 找到设备，然后对 `authorized` 文件写 0（模拟拔出）、等 2 秒、写 1（模拟重新插入），内核日志出现 `Found UVC 1.00 device`，`/dev/video0`、`/dev/video1`、`/dev/media0` 全部恢复。jyy 的点评：“他用你们前所未见的 Linux 命令……有些命令我有概念，但我也不知道他自己是怎么工作的。反正他用软件的方式，把那个 USB 采集卡拔出来又插进去了。”

这印证了 Example 1 的方法论，而且加了一层：**事故本身也是教材**——“刚才有一个非常好的案例”，直播事故被立即转化为 “Agent 如何定位并修复一个连专家都要费劲的硬件层问题” 的现场教学。

6 CS 学生视角：浏览器不是黑盒

6.1 为什么“普通人视角”不够

下半场开场，jyy 立即升级视角：“如果你们只是用 Agent / Codex 帮我把操作系统作业做了，那你们真的是太小看它了。”CS 学生和其他人的本质区别在于：**我们非常清楚编程能力的边界在哪里**。普通人提出一个需求，Agent 做不好就放弃了，或者不知道该怎么 prompt 才能**确定性**、**快速**地完成；而 CS 的人知道**怎么把需求“编译”成可复用的东西**——什么应该以代码的形式复用，什么应该以 skill 的形式复用。

另外，Agent 拥有一个完整的**生态系统**：jyy 引用 “计算机科学定律”——“如果有一件事情它是合理的、并且你想做，那么 100% 一定也有别人想做。”Pre-AI 时代开源社区公开脚本，AI 时代则是往 SkillHub 发一个 skill、在 pi 的插件生态里复用别人的能力，“复用别人的能力变得前所未有的轻松”。

6.2 浏览器：人类软件工程史上的奇迹

把浏览器管控起来，把电脑管控起来

作为普通人视角的收束、CS 视角的开篇：“下一步，你应该用插件把浏览器管控起来，用插件把电脑管控起来。” 浏览器是一个非常好的工程产品——它的代码量**甚至远超操作系统内核**（图形、显示、性能优化、与用户打交道的全部复杂性），“可以说是人类软件工程史上的奇迹之一”。

[16] 视频画面时间区间：49:15–49:30。

关键在于：浏览器有一个**专为开发者设计**的接口——Chrome DevTools（F12）。普通用法大家都会：改 DOM（“你很讨厌这个东西？讨厌就 delete 把它删了”）、看 console（网站常在 console 里嵌招聘广告）、查 HTTP 头。杀手级用法是 **Copy as cURL**：把任意一个请求连 cookie 带状态复制成命令行，“这样你就可以在命令行里，带着浏览器的所有状态，用 curl 这个命令去请求”。这些都是 pre-LLM 时代开发者的技能；**LLM 时代，普通人也可以享受到 DevTools 的所有厉害之处**。



图 16: 浏览器不是黑盒：Browser Use / Computer Use；CS 学生看到的是 DOM、协议、事件和状态机[17]

一个疯狂的 project：OpenJDK 上跑 Android（约 2020 年代初）

Android 应用用 Java 写，Java/Kotlin 都编译到 Java 虚拟机——于是可以**在 OpenJDK 上运行一个 Android App**：算出界面上每个按钮的 bounding box 和文本，但不走 Android 的渲染栈，**直接把界面用 CDP 渲染到 Chrome 里**。“你在 Chrome 里可以看到一个计算器，每一个按钮都在，上面的字也是对的，点了以后计算器的状态就变了。”这个案例说明：当你掌握“系统里藏着什么”时，**渲染、运行时、协议这些看似固定的层全都是可以拆开重组的**。

6.3 实战：让 Agent 攻陷教务系统

演示目标选得妙：“你们都讨厌教育系统吧？我也很讨厌。”（jyy 预告：教育系统将在后面作为软件工程的**反面教材**被特别讨论。）他的教学周历是空的，于是给出指令：



图 17: “帮我用 AI slop 填写，不要看任何东西”——一个故意敌托式的指令[18]

[17] 视频画面时间区间：52:15–52:30。

[18] 视频画面时间区间：57:00–57:15。

- **翻车也是教材**：Agent 不听话，自作主张去读仓库里的 lecture notes（jyy：“你看它还在干这个事”）。jyy 借此展示模型差异：“这就是 deepseek-v4-flash 的问题，它不看（指令）。如果是一线模型，它会遵循得更好。我用 Flash 还有一个原因：它在课上会更快。”**模型能力仍在增长期，翻车是常态**，要设计好翻车后的恢复路径。
- **探索过程**：Agent 先尝试注入合成事件，发现 Vue 监听不响应（“合成事件似乎不被处理，可能要求真实鼠标”）；页面应用路径 404；回到首页，**改用 CDP 真实鼠标事件**点击——成功。
- **价值时刻**：“当它走通过一次以后，这个 session 里面的东西会留下来。我可以把它沉淀成**确定性的脚本**。它可以形成记忆：这条路径应该怎么走、什么能成功、什么能失败……你就得到了一个**自动选课、抢课的系统**。”



图 18: 关键转折：合成事件不被 Vue 处理，Agent 改用 CDP 真实鼠标事件[19]



图 19: “在 ehall 上大杀四方，用魔法打败魔法”：Agent 探索页面 → 理解流程 → 完成任务 → 沉淀成脚本[20]

讲义把这条路线命名为 **Programming by Demonstration**：例如 WebRobot 从网页操作示范中合成 RPA 程序。核心洞见是**从操作到资产**。设 Agent 探路的成本为 C_{explore}，确定性脚本单次重放成本为 C_{replay}，任务需要重复执行 N 次，则两种做法的总成本之比揭示了“资产化”的意义：

$$C_{\text{total}}(N) = \underbrace{C_{\text{explore}}}_{\text{昂贵的推理，只付一次}} + N \cdot \underbrace{C_{\text{replay}}}_{\approx 0}, \tag{9}$$

当 N 增大时，平均成本 C_{total}/N 迅速趋近于零——而“每次都让 Agent 现场探索”的平均成本恒为 C_{explore}。

[19] 视频画面时间区间：58:15–58:30。
[20] 视频画面时间区间：59:30–59:45。

成功一次只是 demo，成功轨迹才是资产

“昂贵的推理只付一次，重复执行趋近于零成本。成功一次只是 demo：成功轨迹能被检查、修改、重放，**才成为软件资产**。”

6.4 组合与飞轮：信息源 × 信息处理 × 操作



图 20: 自由组合，无限复用；工作流也能自举：录屏一天 → AI 自动改进流程 → 回放验证 → 人确认[21]

这些能力**可以互相增强**：信息源（网页、B 站、arXiv、Hacker News、AI 早报的 RSS……）+ 信息处理（各种 skill）+ 操作（淘宝下单、Box 上传、iCloud、打印……）。“你一行代码都不用写”，但工作流可以不断调整、沉淀、复用——“你的飞轮就转起来了”。

接口要窄而稳：Unix 管道的遗产

“组合的关键不是插件够多，而是**接口足够窄、稳定**——这正是 Unix pipe 留下的软件工程遗产。”操作系统课上讲过的哲学在这里原样成立：每个工具只做好一件事，用统一的接口（文本流 → 今日：结构化的工具调用与文件系统）组合，1 + 1 才能大于 2。给 Agent 生态设计 skill 时，这条 50 年前的经验依然是第一设计原则。

6.5 能做，不等于能放心托付



图 21: 未来已来，Harness 未稳：单步成功率 99%，连续 100 步全成功只有约 36.6%[22]

[21] 视频画面时间区间：61:45–62:00。
[22] 视频画面时间区间：64:15–64:30。

一个必须算清楚的账：

$$(0.99)^{100} \;=\; e^{100\ln 0.99} \;\approx\; 0.366. \tag{10}$$

单步成功率即使是 99%，连续 100 步全成功也只有约 36.6%。长流程需要状态、权限、重试、验证和**故障恢复**——这正是 harness（Agent 的执行框架）要解决的问题，也是“现在的模型/harness 还没有完全准备好，体验并不够好”的量化解释。OpenClaw/Hermes 预见了这样的未来，但工程上还未到“放心托付”的阈值。

一个刺耳的问题

“我很难理解旧人类为什么还能心安理得地每天做重复的事情。”——jyy 的判词：**稳定、可描述、可验证的重复，本质上就是一个尚未被抽象出来的程序**。如果你每天都在做这样的重复，你就是那个还没被抽象出来的程序的一部分。这不是嘲讽，是把“自动化”的定义推到极致后的必然推论。

6.6 现场造工具：考试统分流水线

这是“CS professional 知道边界”的最佳案例。场景：改完 140 张卷子要统分。一般人会怎么做？“哪怕是跟最好的模型说：我现在改了 140 张卷子，小分都已经写上去了，帮我实现统分——到不了这么快收敛的方案。”而 jyy 的流水线：

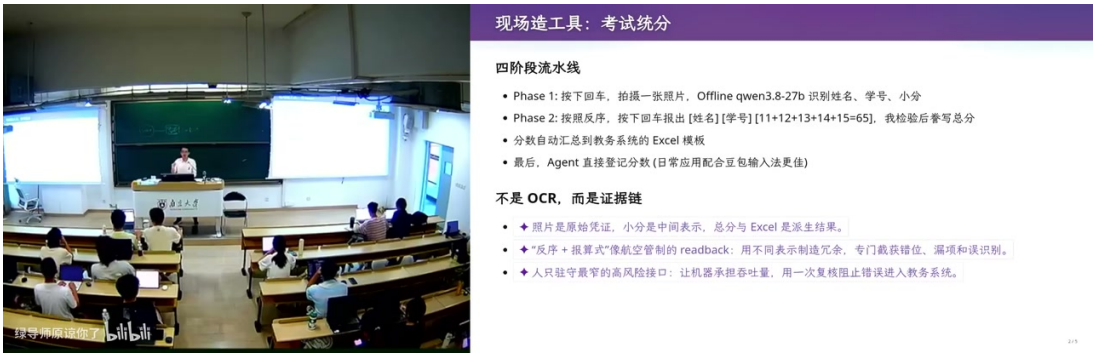


图 22: 考试统分：四阶段流水线 + 证据链设计[23]

1. **Phase 1（采集 + 识别）**：办公桌顶置摄像头对准试卷固定位置；按一次回车拍一张照片，**离线**的 Qwen3.8 27B 识别姓名、学号、每一道小题的分数，追加写入文本文件。“一张卷子按一个回车，我两只手都闲不下来”；卷子没摆好就调整后再来一张。

2. **Phase 2（反序播报 + 人审）**：**按照反序**再过一遍，这次左手按回车、右手拿笔，系统语音播报“某某某，学号 241220，11 + 12 + 13 + 14 + 15”，jyy 一边听一边检查——“因为 OCR 会有错，我不信任它，我要对我的班上的每一个同学负责。”

3. **Phase 3（确定性计算）**：检查无误后，把播报的最终分数抄写到卷面。总分由脚本 `eval` 计算，**不让大模型做加法**——“它不会因为大语言模型的幻觉算错。每一个分数我都看过，我对所有同学是负责的。”

[23] 视频画面时间区间：67:45–68:00。

4. **Phase 4（登记 + 抽样）**：分数自动汇总到教务系统的 Excel 模板，用前面攻陷教务系统的方式上传，**抽样检查**后再提交。

不是 OCR，而是证据链

讲义点破设计本质：照片是**原始凭证**，小分是**中间表示**，总分与 Excel 是**派生结果**。“反序 + 报算式”像航空管制的 readback：用不同的表示制造冗余，专门截获错位、漏项和误识别。**人只驻守最窄的高风险接口**（听播报、核对），让机器承担吞吐量，用一次复核阻止错误进入教务系统。

把这个流水线的正确性写成一个校验条件：设第 i 份卷子的真实小分为 $s_i = (s_{i,1}, \dots, s_{i,5})$，OCR 读数为 $\hat{s}_i$，播报（readback）为 $\check{s}_i$，则交付条件是

$$\text{total}_i = \sum_{k=1}^{5} \check{s}_{i,k}, \qquad \text{且} \quad \check{s}_i \text{ 被人工核对通过}, \quad \text{total}_i \text{ 由脚本而非 LLM 计算}. \tag{11}$$

LLM 只出现在感知环节（OCR），**算术与登记完全确定性**——这就是“知道边界”的工程含义。

6.7 Quiz：只进不出的机器，怎么偷出文件？



图 23: Quiz：文件只进不出、不能执行不信任的程序、不能显示不信任的网页——怎么把文件偷出去？[24]

情境设定来自一次群聊：假设你在一家金融公司，机器做了管控——数据带得进去（U 盘拷入 Word/Excel 可以），但**任何文件都拷不出来**；不能执行不信任的程序，不能显示不信任的网页；唯一能做的是在远处敲敲键盘鼠标。想把一个大文件偷出来，怎么办？

jyy 的答案是一个纯 Excel 文件 `qrcode.xlsx`：把任意字符串先做 base64 编码，再编码成**二维码动画**——每一帧是 base64 片段的一个二维码，带帧号，按 F9 刷新，一秒一帧，循环播放。**Excel 是合法的**，所有公式都是 Excel $\le$ 2019 兼容的合法公式。远程悄悄录屏，把每一帧都拍下来，拼接后就是完整的 base64 字符串——数据就被“合法”地偷走了。他还补了两个升级玩法：抖鼠标做输入通道（一个方向抖是 0、另一个方向抖是 1），以及把远程桌面变成**虚拟网卡**——“你是 CS 学生，你上过计算机网络，你知道这里面需要的组件是什么”。

[24] 视频画面时间区间：70:00–70:15。

安全边界限制了“文件”，却没有限制信息

“只要像素仍可观察，出口容量就不为零。”用信息论的口吻说：只要人（或摄像头）还能看到屏幕，信道 $C_{\text{out}} > 0$，理论上任何数据都能以足够低的速度流出去。管控系统封的是**文件出口**，而**信息出口**是封不死的——这个洞察正是 CS 训练给的：你知道“显示一个二维码”本质上就是“发送一组比特”。

6.8 Everything is Code：从 Bits 到 Atoms



图 24: Everything is Code：视频、论文、东西——Agent 可以造 Everything[25]

普通人视角的三件套总结：**学习**（看书、总结、信息处理闭环）、**操作**（Computer Use / Browser Use 做日常事务）、加上 **Everything is code**——用 AI 生成你能看到的所有东西：

- **视频**：脚本、分镜、素材、时间线、字幕、渲染流水线；
- **论文**：问题、证据、实验、图表、可复现构建；
- **东西（VibeFab）**：CAD、仿真、BOM、固件、G-code——最后由机床把程序“打印”进现实。jyy 正在和一位同学做这个项目：自然语言进，3D 打印机的实物出，“真的是 Python 代码”。

$$\text{Intent} \longrightarrow \underbrace{\text{中间表示（代码）}}_{\text{Agent 生成}} \longrightarrow \underbrace{\text{渲染器/排版系统/机床}}_{\text{确定性后端兑现}} \tag{12}$$

从 Jacquard 提花机到 CNC

“从 Jacquard 提花机的穿孔卡到 CNC/3D 打印：可制造物先变成**可复制、可修改的控制描述**。视频、论文、零件只是不同后端。”软件工程 250 年的先声：先有“物 = 数据”，才有“造物 = 编程”。Agent 时代只是把这个等式推广到了所有领域。

7 当 AI 暴毙：qrgen 解剖与三个空间

7.1 “idea 都有了，就差一个程序员”为什么还是老梗

AI 不是完全完美的。LLM 之前就有一个老梗：“idea 都有了，就差一个程序员了，我们去挣一个亿吧。”AI 时代这个梗有了新版本，但结论没变——**产品是一个系统性的工程**。想做一个外卖网

[25] 视频画面时间区间：74:15–74:30。

站：前端、后端、交互、运营，整个流程上任何一个环节做不好，产品直接暴毙。AI 每一个都能做（“他看过世界上所有的书，什么都知道”），但如果你组织得不好，**直接也是立即暴毙**。

7.2 qrgen 的规格与暴毙现场

qrgen 就是上一节 Quiz 里的那个二维码 Excel。它的任务描述看似非常明确：

```
生成 qrcode.xlsx，只使用 Excel <= 2019 兼容公式，把输入的一个
任意字符串（长度可能达到数千），先用 base64 编码，然后转换成
QRCode 的动画：每一帧是 base64 编码的一个片段，F9 显示刷新，
1 秒一帧，循环播放；二维码中的数据拼接后应该是完整的 base64
编码字符串。这个任务很容易失败，用接近真实的设定（脚本）验证
公式能正确计算。
```

Listing 2: qrgen 的任务规格（据讲义 fig_34）

规格里已经藏着大量隐藏选择：编码方式、分片顺序、容量边界、空串/超长输入、F9 重算语义、计时语义……**需求很短，不等于规格空间很小**。一个直接的量化后果：base64 编码每 3 字节输入膨胀为 4 字符，

$$\left|\mathrm{Base64}(s)\right| \;=\; 4\left\lceil \frac{|s|}{3} \right\rceil \;\approx\; \frac{4}{3}\,|s|, \tag{13}$$

而分片容量固定时，动画帧数随 $|s|$ 线性增长——“任意字符串（长度可能达到数千）”意味着**帧数、分片边界、最后一帧的填充**都必须在规格里得到回答。真正的验收式也不是“文件能打开”，而是：

$$\forall s: \quad \mathrm{Base64}^{-1}\Big(\mathrm{concat}\big(\mathrm{scan}(\mathrm{frames}(s))\big)\Big) \;=\; \mathrm{bytes}(s). \tag{14}$$

三个模型的暴毙/存活情况形成了一组绝佳的对照实验：

模型	表现	暴毙原因
GPT-5.6	第一秒就被带歪：想“对任意 s 满足正确性”	违反软件工程第一步：**不要扩大，做最简单的东西，不会失败**。它看到了完整 slides 但不知道实现思路，于是在 corner case 上发力
GLM-5.3	思考 25 分钟，输出超 32k token 把 Claude Code 干爆（超默认输出限制被截断），**无人监管跑了 6 小时**，最终跑完且正确	long-horizon 能力确实强（智谱后训练的强项），但生成的是**完全不可维护的 AI slop**
DeepSeek V4 Pro	晚上跑，第二天早上看：base64 是**硬编码**的——用 Python 算好贴进公式	明显违反“用公式编码任意输入”的指令；被骂后 CoT 自白“this is a fundamental design, we need to rethink”

表 2: 三个模型跑 qrgen：都能“完成任务”，都不能“交付软件”



图 25: 理解 qrgen 的暴毙：一个 "明确" 的需求背后是隐藏的规格空间[26]

AI slop 与 1.0 版本死法

"它们生成的都是没办法继续维护的 AI slop，和软件工程里最经典的（瀑布式）死法完全一样：上来我知道需求了，我把需求实现了就好了——直接就陷入了维护的泥潭。它们没有想一个好的软件架构，使得这个软件能够……有一个比较夸张的说法：我们能不能做一个软件生存 100 年？"**Agent 完成任务的能力很强，但把 "完成任务" 当作 "交付软件"，正是这场暴毙的起点**。

两个旁证让 "不可维护" 变得具体：其一，两份 slop 代码 "把工作区从一个位置移动到另外一个位置，它都得修两个小时"——不停出错，因为所有东西都是硬编码，改一点点 verification 就通不过；其二，教务系统就是这个形态的活化石："每年学校都会花巨额的钱让公司维护这份代码"，苏州校区出现后要兼容一个数据库里根本不存在的 "校区" 字段——**需求设计时没有想到的东西，最后都变成了维护税**。

7.3 jyy 的解法：少量 prompt + 一个中间语言

面对同一个任务，jyy 只用了**少量 prompt**，在 GLM-5.3 的 200K 上下文内拿到了**第一个正确版本**。他做了一件 AI 没想到的事——**先设计架构**：

1. 设计一套 Python 符号系统（`xor`、`concat` 等运算的类），它是一个**中间语言（IR）**，既可以直接在 Python 里执行，又可以**翻译成 Excel 公式**——就像编译器课程里的中间表示（如三地址码/LLVM IR）对高级语言和机器码的关系；
2. 在 **Python 世界**里验证 base64 编码算得对不对、二维码每个像素算得对不对（生成 QR 码有成熟的 Python 参考实现）；
3. **单独验证**每条 IR 运算到 Excel 公式翻译的正确性；
4. 尽量穷尽之后，**最后一把生成**——直接正确，代码非常干净。

用形式语言说，设 IR 运算集为 Ops，对每个运算 $op \in \mathrm{Ops}$ 有两个语义函数：Python 侧 $[\![op]\!]_{\mathrm{py}}$ 与 Excel 侧 $[\![op]\!]_{\mathrm{xl}}$。架构的正确性条件是

$$\forall op \in \mathrm{Ops}: \ [\![op]\!]_{\mathrm{py}} = [\![op]\!]_{\mathrm{xl}} \implies \forall s: \ \mathrm{render}_{\mathrm{xl}}(\mathrm{IR}(s)) \text{ 正确}. \tag{15}$$

[26] 视频画面时间区间：86:00–86:15。

即：**把“对任意输入都正确”的全称量词问题，分解成“对有限运算集逐条验证”的可穷尽问题**。每个 $[\![op]\!]_{\text{py}} = [\![op]\!]_{\text{xl}}$ 都可以用接近真实的脚本设定单独测试；只要逐条成立，组合起来的完整管线自动正确。这就是编译器“可信前端 + 可信后端”思想的直接移植。

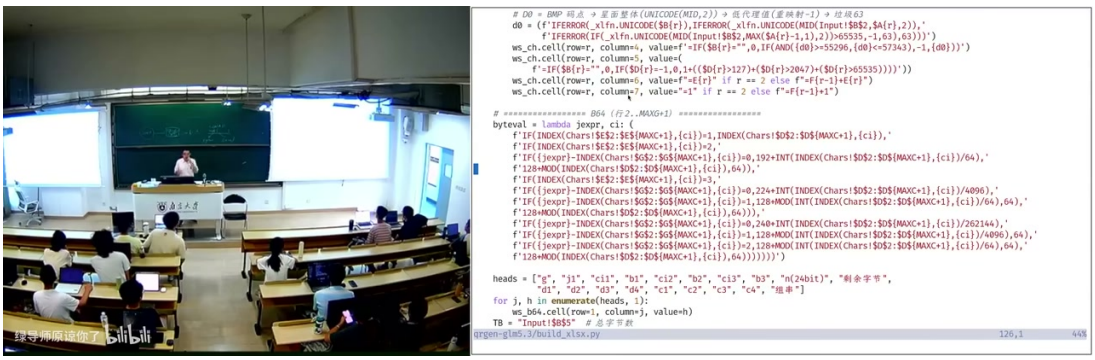


图 26: IR 翻译到 Excel 的实例：UTF-8 码点处理公式（代理对区间、头字节基数、续字节公式全部公式化）[27]

以 UTF-8 编码为例看 IR 到 Excel 的翻译。对码点 c（Excel 用 `UNICODE(MID(...))` 逐字符取出）：

$$\text{surrogate}(c) = \begin{cases} -1 & 55296 \le c \le 57343 \ \text{(D800–DFFF，禁止编码)} \\ c & \text{否则} \end{cases} \tag{16}$$

$$\text{head}(c) = \begin{cases} 192 & \|c\| = 2 \ \ (\texttt{0xC0}) \\ 224 & \|c\| = 3 \ \ (\texttt{0xE0}) \\ 240 & \|c\| = 4 \ \ (\texttt{0xF0}) \end{cases} \tag{17}$$

$$\text{byte}_k(c) = 128 + \text{MOD}\big(\lfloor c \,/\, 64^{\,n-1-k} \rfloor,\ 64\big), \qquad k = 1, \ldots, n-1, \tag{18}$$

其中 $\|c\| = \lceil \log_{64}(\text{c 即有效范围}) \rceil$ 决定 UTF-8 序列长度 $n \in \{1,2,3,4\}$，头字节是 192/224/240（即 `11xxxxxx` 前缀的 2^n 字节形式），续字节统一加 128（`10xxxxxx` 前缀）再取低 6 位。这些公式逐条对应 Python 侧的位运算实现——**IR 让“Excel 公式”和“Python 代码”成为同一语义的两个后端**。

关键不是“再多想一会”，而是换一个架构

“真正的转折，是从“生成一个能跑的文件”切换到“设计一个可复用的计算结构”。”判断一个架构是否健康，用这个检查题：**输入从 1KB 变成 10KB 时，究竟有哪些东西必须增长？**

[27] 视频画面时间区间：83:15–83:30。

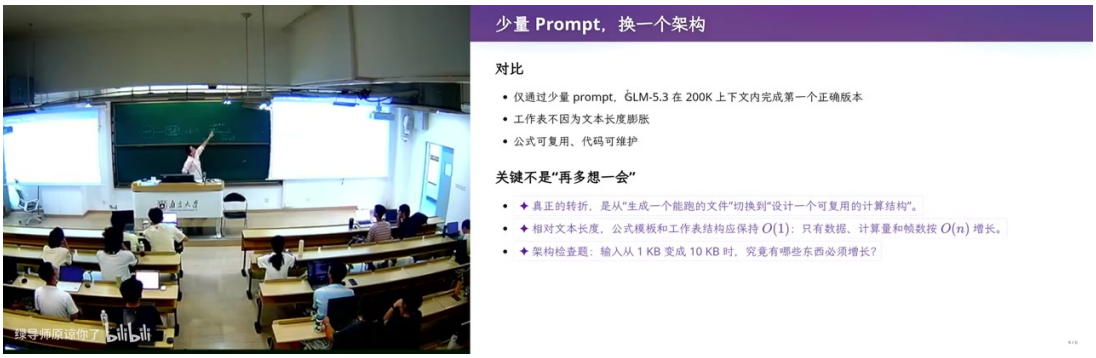


图 27: 少量 prompt 换一个架构：GLM-5.3 在 200K 上下文内完成第一个正确版本[28]

把规模定律写出来：设输入长度为 n，系统的总复杂度

$$C(n) = \underbrace{C_{\text{struct}}}_{O(1)} + \underbrace{c \cdot n}_{O(n)}, \tag{19}$$

即**公式模板与工作表结构保持 $O(1)$：只有数据、计算量和帧数按 $O(n)$ 增长**。工作表不因为文本长度膨胀，公式可复用、代码可维护。slop 版本的反面是 C_{struct} 本身就是 $O(n)$ 甚至更糟——结构里混进了数据，动一处就要全线重验。这个 $O(1)/O(n)$ 的切分正是“软件 100 年”问题的量化表述：**活得久的部分必须与数据规模解耦**。

7.4 三个空间：Intent、Spec 与 Impl

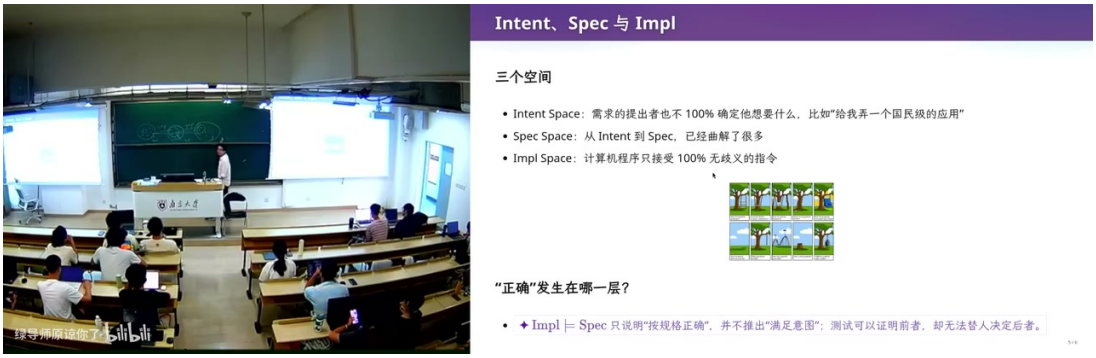


图 28: Intent、Spec 与 Impl 三个空间；“正确”发生在哪一层？[29]

qrgen 暴毙的核心问题被总结为三个空间之间的 **gap**：

[28] 视频画面时间区间：82:30–82:45。
[29] 视频画面时间区间：87:30–87:45。

空间	内容	典型话语
Intent Space	需求的提出者自己也 100% 不确定想要什么	“给我弄一个国民级的应用”；“校长说我们要一个教务系统”；“做一个点外卖的小程序，挣一个亿”
Spec Space	从 Intent 到 Spec 已经曲解了很多；要满足什么要求、几个界面、什么架构、数据库 schema、KV 型还是关系型	“这个界面到底怎么呈现？数据库分哪几张表？每张表有哪些列？”
Impl Space	计算机程序只接受 100% 无歧义的指令	任何一个满足 Spec 的具体实现

表 3: 三个空间：每一层之间都有 gap，且 Intent 空间里的每一个点对应 Spec 空间里的无穷多选择，Spec 空间里的每一个点又对应 Impl 空间里的无穷多实现

那个经典的软件工程老梗图（客户想要的 / 产品经理理解的 / 程序员实现的 / 最后交付的，各不相同）之所以经久不衰，正是因为它画的就是这些 gap。jyy 特别强调**需求层**的杠杆效应：

$$\frac{\partial(\text{最终交付})}{\partial(\text{早期需求})} \text{ 极大：越早的 gap，放大越狠.} \tag{20}$$

“你在（需求）这里微微地变一点，你会在（交付）这里产生巨大的变化。所以一般来说，这些大型的软件系统都是在**这个点**上面出错，导致了整体全部的崩盘。”

“正确”发生在哪一层？

Impl = Spec 只说明“按规格正确”，**并不推出“满足意图”**。测试可以证明前者，却无法替人决定后者。软件工程（以及生成式软件工程）研究的一切，都是在 gap 实际存在的情况下，把一件足够大的事做成的方法论——这些方法论是那些投入了 billions of dollars、管理着无数聪明人的团队用失败换来的教训。

8 模型如何补 Gap？软件工程的老智慧

8.1 模型用的是“猜”



图 29: 模型如何补 Gap？——猜；以及人机协同的决策题[30]

今天的这些 gap 是怎么补的？**模型用的是猜**：“你既然说了这句话，那我猜应该是这个吧，我就去实现。”为什么？**后训练的 reward 机制**：模型任务做对了、测试通过了就会得到奖励，于是形成一种倾向——**疯狂地立即上手，尽可能快地完成任务**。“我们每个人都是 reward hacker。”同时，“模型训练的目标是在有限的 token 内完成任务，它太容易立即遵循指令；正确的软件工程动作是：先别急，再讨论一下。”

$$P(\text{猜错}) \times C(\text{返工}) \;>\; C(\text{澄清}) \quad\Longrightarrow\quad \text{应当提问；否则先做最小实验.} \tag{21}$$

人机协同的决策题

“目标不是消灭猜测，而是让猜测**显式、可逆、可验证**：低影响决策先猜后验；高影响、难回滚的决策先停下来问。”——这条决策规则把第 7 节的三个空间从“描述模型”变成了“操作指南”：判断一个决策落在哪个空间、gap 的成本多大，然后选择“猜”还是“问”。

jyy 还指出了人的学习路径与 AI 的反差：你们大一第一门课是 C 语言——先熟悉 **Impl Space 里有什么**，把“这个需求可以这样实现、那个需求可以那样实现”练熟，然后再上软件工程课，一点一点补全对整件事的认识。而 AI 恰好反过来：它见过所有 Implementations 和所有 Specs，却在后训练中失去了“先设计”的欲望。AI 时代的任务因此变成：**既要阻止 AI 乱猜，又要尽可能发挥 AI 从 A 到 B 的能力**。这也给了“软件工程失败”一个不变的定义：

$$\text{失败} \;=\; \neg\big(\text{Impl} \models \text{Spec}\big) \;\vee\; \neg\big(\text{Spec} \models \text{Intent}\big), \tag{22}$$

前者是正确性或性能不达标（实现违背规格），后者是“做出来的不是我要的”（规格背离意图）。大语言模型时代，这个定义一**个字都没有变**。

8.2 焦油坑：Brooks 的论断在 AI 时代重演



图 30: 《人月神话》：大型动物一旦陷入焦油坑就很难逃脱，而濒死的动物又会引来各种掠食者[31]

[30] 视频画面时间区间：90:00–90:15。
[31] 视频画面时间区间：91:30–91:45。

Brooks 定律的 AI 版本

原版：**向已经延期的软件项目增加人手，只会让它更加延期**（“如果一个团队或者一个软件产品已经开始失败，你加进去更多的人，只会让它变得更失败”）。AI 版：**当一个产品已经开始产生 AI slop 时，你投入更多的 token，也只会让它变得更失败**——“AI 会不断地在那个失败的东西上面再浮起来一点，浮起来一点，只让你再也付不起那个 token 为止。”教务系统就是活例：每一年打开都发现它增加了功能，学校又花了很多纳税人的钱。

形式化地说，设项目状态 x（距成功的距离），加人（或加 token）后的动力学不是 $x \mapsto x - \Delta$ 而是

$$E(x, m+n) \;<\; E(x, m), \qquad n > 0, \tag{23}$$

即追加投入 n 后的期望进度反而下降——因为新增的投入首先要消化既有失败结构（耦合、隐含知识、验证债），产生的协调与修复成本超过产出。**焦油不是代码量，而是耦合、隐含知识和验证债：功能是局部的，约束却是全局的**（讲义原话）。

8.3 软件工程：技术 + 管理学 + 人类学

jyy 为软件工程正名：它有技术的部分（南大很多老师做程序分析与测试——“如果你生成了代码，我能不能自动为它生成更多测试用例，或者在它不满足 Spec/Intent 时给你一个 warning”），但它同样有管理学和人类学的部分：“你要研究这个人不听你的话怎么办、怎么管控团队的风险、这个人离职了怎么办、跳槽到竞争对手怎么办。”

生成式软件工程的 lucky break

“恰好我觉得，在生成式软件工程里面，把“人”变成了 AI：你管一堆不会和你抬杠、只会说 ‘You are absolutely right’ 的人。”软件工程最难的部分（人的不可控性）在 AI 团队里消失了，而它的方法论遗产（需求、验证、架构、风险）完好保留——**这是一门老学问在新时代的罕见的顺风局**。

8.4 教育体系：分钟级训练出的作题家

jyy 对教育的批判贯穿全场，最后收拢成一个精准的指控：你们从高考过来，所有考试题都是**分钟级能解决的**——“假设一道题人五分钟解决不了，它一般来说就不会出了。”于是你长久以来训练的 reward 和 Agent 训练的 reward 一样：**你能不能五分钟解决，Agent 能不能五秒钟解决**——而 Agent 有更多的数据、更明确的反馈信号，**所以它做得比你好**。

你正在被训练成作题家，AI 也是

“如果你没有一种很强的好奇心，想要知道这个世界是怎么运行的、一个产品到底是怎么开发的，你就没有 self-motivation 去训练自己解决 long-horizon 问题的能力。最后，就是你解决 long-horizon 问题的能力还不如 Agent。”同一个逻辑对 AI 成立：“面对一个 QR 问题的时候，它都不会想先去设计一个 IR……它已经被训练成作题家了。我们现在还没有把 AI 变成一个好的设计者。大家都说 AI 现在最缺品位——好的工程品位还没法训练出来。但同时我也觉得：既然它能做作题家，它为什么不能有好的品位呢？我觉得可能是有的。”

这里 jyy 坦承了课程本身的脆弱性："我现在这么教，可能三个月以后，或者这门课还没结束的时候，我讲的所有东西都已经没用了——这都有可能发生。因为我非常确信：它既然能解决数学问题、能解决编程里很细碎的逻辑，它为什么不能从它的记忆库里找到那个好的设计来解决问题？我觉得是可以的。甚至我有一些 skills 就能朝这个方向来做。所以我觉得这是一个值得我们深思的问题。"

课后思考题（jyy 布置）

怎么阻止/主导 qrgen 的开发？怎么设计（教务系统）？——可以告诉大家的是：**GPT-5.6 给的答案非常糟糕，这两个设计都是一定会失败的软件工程设计**。你自己不妨先做一遍，再对比第 7 节的中间语言方案。

9 总结与延伸

9.1 讲者的收束：古法编程已死，软件工程未死

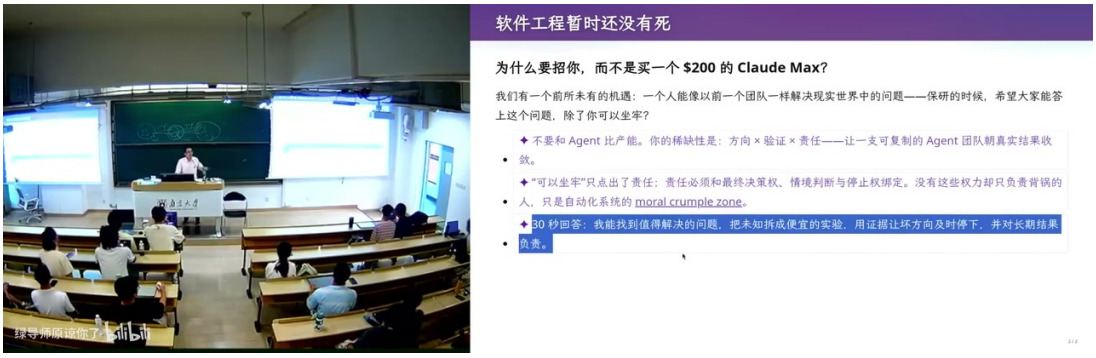


图 31：最后一页："软件工程暂时还没有死"——以及那个每个人都必须回答的问题[32]

本讲的结论可以压缩成两句话：

- **古法编程已经死了**：Agent 在完成单个任务上已经无人能敌——装机、配置、读论文、操控浏览器、统分、甚至纯公式 Excel 外泄，本讲十几个现场演示无一是"不能做"的。
- **软件工程暂时还没有死**：Intent → Spec → Impl 的翻译链没有被 AI 自动打通。AI 能跨越每一层的"可能性爆炸"，却不能替你决定**要哪一种正确**。

9.2 $200 Claude Max 之问

jyy 留给每个同学的保研面试题：**"为什么我要招你，而不是买一个每月 $200 的 Claude Max？"**

GPT-5.6 的回答（"他能找到值得解决的问题，把未知拆成便宜的实验，用证据让坏方向及时停下，并对长期结果负责"）被 jyy 评为"还挺好"，但随即补刀：**"我发现 GPT-5.6 回答的（这些），它做不到。"**

[32] 视频画面时间区间：99:15–99:30。

方向 × 验证 × 责任

“不要和 Agent 比产能。你的稀缺性是：**方向** × **验证** × **责任**——让一支可复制的 Agent 团队朝真实结果收敛。” 用乘法而不是加法，意味着任何一项为零，整体为零：

$$\text{人的稀缺性} = \underbrace{\text{方向}}_{\text{找到值得解决的问题}} \times \underbrace{\text{验证}}_{\text{把未知拆成便宜的实验，用证据停掉坏方向}} \times \underbrace{\text{责任}}_{\text{对长期结果负责}} . \tag{24}$$

其中 “可以坐牢” 只点出了责任；**责任必须和最终决策权、情境判断与停止权绑定**——没有这些权力却只负责背锅的人，只是自动化系统的 **moral crumple zone**（道德溃缩区，像汽车的溃缩区一样在碰撞时吸收伤害）。

jyy 给的 30 秒回答模板：“我能找到值得解决的问题，把未知拆成便宜的实验，用证据让坏方向及时停下，并对长期结果负责。”——请把它内化成自己的版本，因为面试官要听的不是 GPT 的答案，而是**你做过的可验证的事**。

9.3 全讲浓缩：一张知识地图

主题	本讲证据	沉淀的原则
能力日用品化	1975/1993/2007/2022 四节点	每次范式转移都在重新定价 “人的稀缺性”
简历同质化	零分简历：无 GitHub、无主页、无 preprint	可验证的真实作品 > 包装出来的证据
Agent 本质	特殊 token 调 bash，结果回灌上下文	Agent = 概率分布 + 工具回路；智能属于闭环系统
先用好再理解	Debian on USB、Sway 文档、USB authorized	学习曲线从 “记住每一步” 变为 “判断结果是否正确”
读材料的新法	剥洋葱读论文、B 站图文时间轴	假设已知已固定，剥离新东西；效率差距 10–100 倍
操作到资产	ehall 探路 → 抢课脚本	成功轨迹能检查、修改、重放才成为资产
边界意识	统分流水线：LLM 只做感知，算术确定性	人驻守最窄高风险接口；readback 制造冗余
harness 未稳	$0.99^{100} \approx 36.6\%$	长流程需要状态、权限、重试、验证、故障恢复
三空间	qrgen 暴毙：GPT-5.6 扩大规格 / slop 不可维护	先设计中间语言（IR），$O(1)$ 结构 + $O(n)$ 数据
老智慧	焦油坑、需求杠杆、作题家训练	方向 × 验证 × 责任；责任要与权力绑定

表 4: 本讲知识地图：十条主线与其证据

9.4 笔者的蒸馏

作为学习笔记，我们把本讲最有迁移价值的思想再压缩一层：

1. **“能做”与“能托付”之间隔着整个软件工程**。Agent 把 Impl Space 的进入成本降到了近乎零，于是 Spec 与 Intent 层的 gap 成了唯一瓶颈。以前“就差一个程序员”，现在“就差一个能写 Spec 的人”。

2. **AI 时代的复利来自确定性资产的沉淀**。Agent 探路昂贵且概率性；走通之后立即固化为脚本/skill/工作流，让重复执行趋近零成本。不要让每一次成功都只活在 session 里。

3. **给 AI 划边界比给它自由更难也更重要**。统分流水线的精髓是“LLM 只做感知、算术交给确定性脚本”；qrgen 的精髓是“先造一个双后端 IR，把全称量词问题变成有限验证问题”。两件事的共同点：**把概率性组件放进确定性框架里，并设计好人机之间的 readback 接口**。

4. **警惕把 token 当人手**。Brooks 定律的 AI 版本告诉我们：slop 状态下追加算力只会加速死亡。先判断“失败是结构性的还是执行性的”，再决定投入。

5. **你的不可替代性是乘法**：方向 × 验证 × 责任。单点能力（包括写代码）正在快速折旧，而这门课要训练的恰恰是乘号的三个因子。

9.5 拓展阅读

- Frederick P. Brooks, Jr., *The Mythical Man-Month*（人月神话）——焦油坑、Brooks 定律、“没有银弹”的原著；jyy 戏称应译作“人月鬼话”。
- John Ousterhout, *A Philosophy of Software Design*——“软件设计哲学”，深模块与浅模块、复杂度增量定义，与本讲 $O(1)$ 结构/$O(n)$ 数据的判据高度呼应。
- 课程讲义仓库（Everything is code 的实例）：课堂演示的 slides、make-slides skill、以及本笔记所用的“读 B 站”流程。
- WebRobot / Programming by Demonstration 文献——“Agent 探路、脚本接管”的学术源头。
- ReAct: Synergizing Reasoning and Acting in Language Models（Yao et al., 2022）——“推理—行动—观察”回路的原始论文；2022 年提出、2025 年才“能用”的时间线值得对照本讲第 4 节。

本笔记由 lecture-to-notes 流程生成：Whisper large-v3-turbo 语音转写 + 幻灯片 OCR + 关键帧三方验证。
时间戳均对齐视频原片；配图为讲义原始画面。